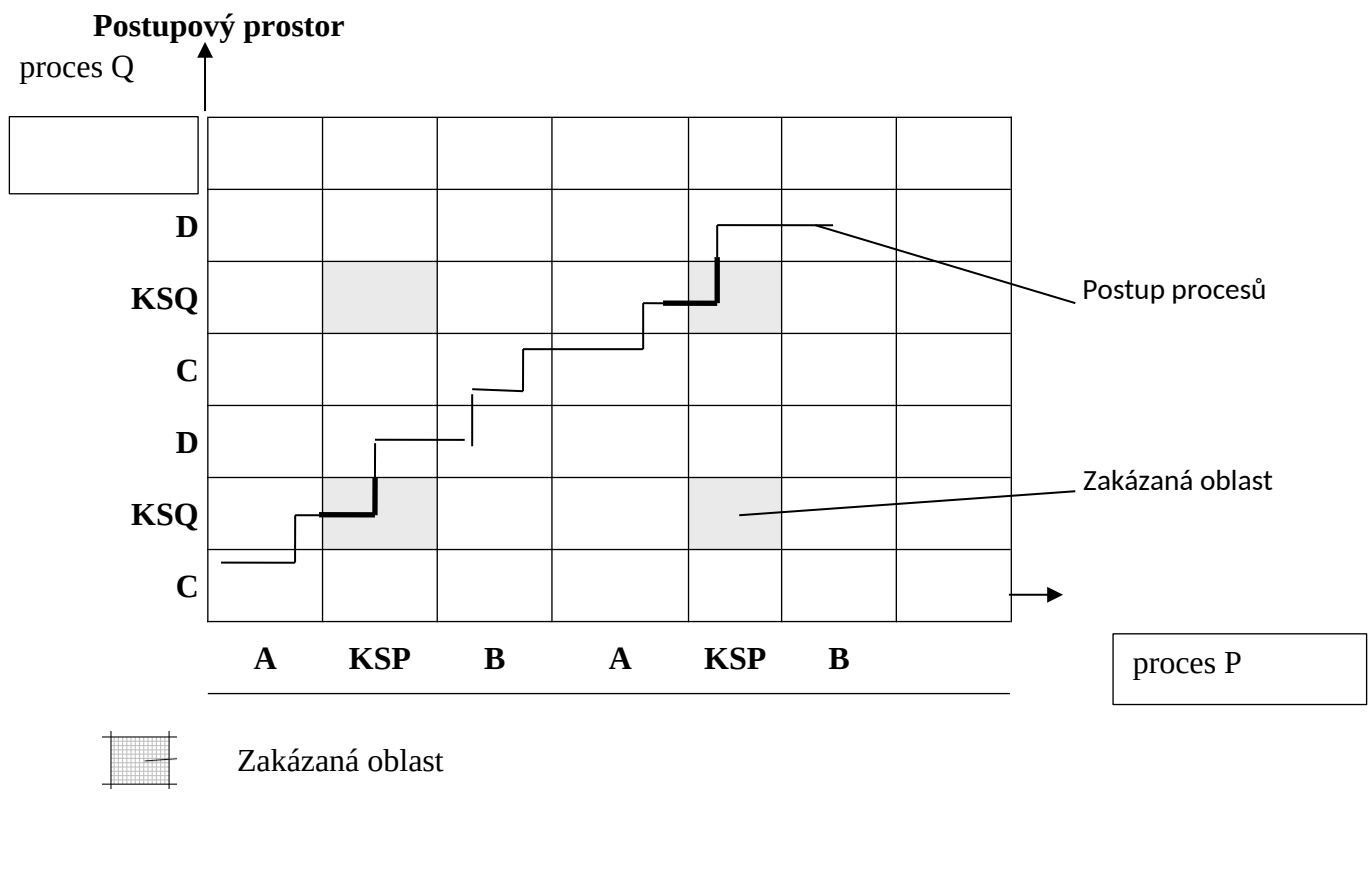


1 Synchronizace procesů

V multiprogramových OS je potřeba zajistit sdílení jednoho prostředku (soubor, paměť) několika procesy. Pokud by totiž dva nebo více procesů například přistupovalo k jednomu souboru s tím, že by více procesů do něj zapisovalo, došlo by ke zhroucení celého systému. Proto se využívá synchronizace procesů, která se těmito kolizím snaží zabránit. Využívá k tomu přerušení v této posloupnosti:

1. Odebrání procesoru procesu - přerušení
2. Procesor se předá druhému procesu
3. Odebrání procesu procesoru – přerušení
4. Procesor se předá prvnímu procesu



obrázek 1 postupová cesta, postupový prostor, zakázaná oblast

P, Q – paralelní procesy

A, B, C, D – příkazy

KSP, KSQ – kritické sekce – použití společného prostředku (disk, tiskárna, proměnná)

Př. Programu

```
cobegin
  --P:
  loop A;
    KSP;
    B;
  end loop;
  --Q:
  loop C;
    KSQ;
    D;
  end loop;
coend;
```

Nejvýhodnějším modelem paralelismu, ve kterém se snadno znázorňuje synchronizace více pro více procesů, jsou **Petriho sítě**. Petriho síť lze zadat orientovaným ohodnoceným multigrafem, v němž se užívají dva typy uzlů: místa (znázorňují se kolečky) a přechody (znázorňují se čárkou kolmou na hrany). Místa a přechody se propojují orientovanými hranami

tak, aby žádné sousední uzly nebyly téhož typu. Přechod mezi místy **p** a **q** je připraven, jestliže každé místo **p** obsahuje alespoň tolik teček, kolik vede z místa **p** do přechodu hran.

Připravený přechod se může provést:

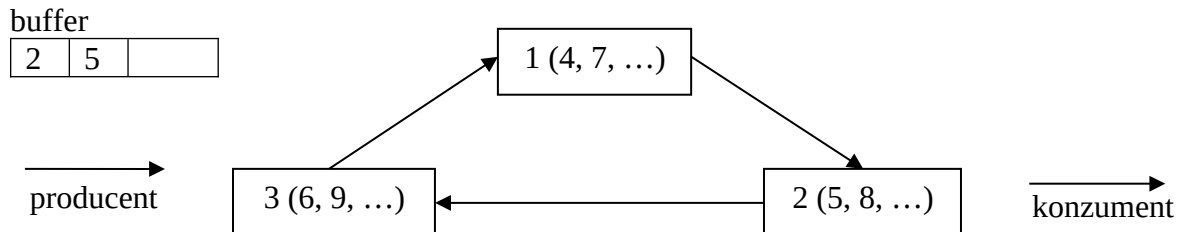
- Z každého místa **p** se odebere tolik teček, kolik vede hran z místa **p** do přechodu.
- Do každého místa **q** se přidá tolik teček, kolik vede hran z přechodu do místa **q**.

Producent – konzument

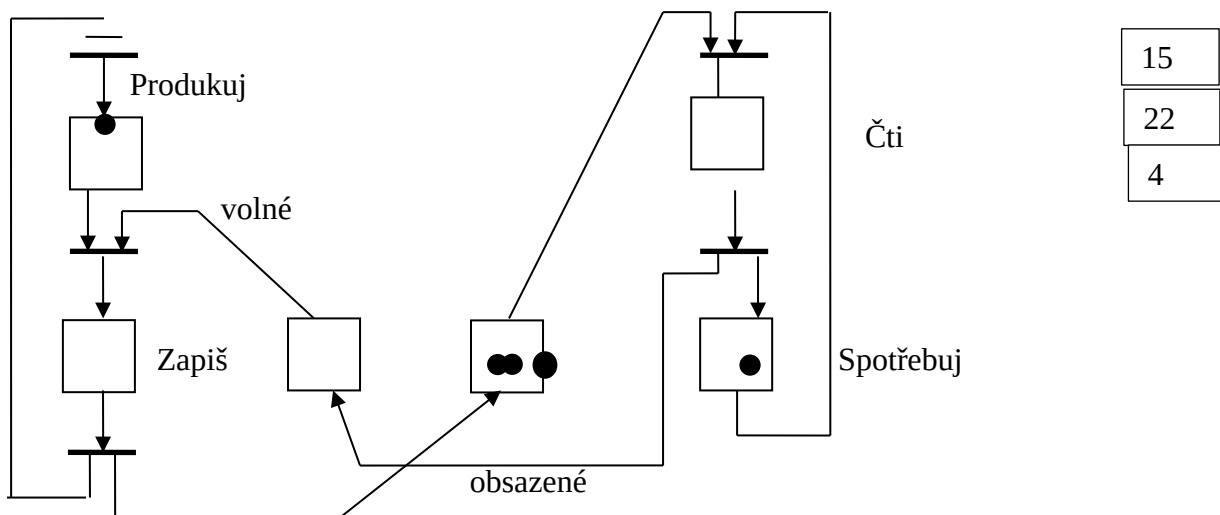
Asynchronní komunikace procesů pomocí společné paměti (asynchronní - procesy na sebe nečekají). Využívá se zásobníku, kam jeden proces posílá data a druhý je z něj čte. V případě většího počtu procesů se zvětší i počet teček v níže uvedeném schématu.

Producent – proces, který produkuje, nemůže produkovat, pokud je zásobník **plný**.

Konzument – proces, který odebírá. Nemůže odebírat z **prázdného** zásobníku.



obrázek 3 producent a konzument



obrázek 4 producent a konzument v Petriho síti pro 3 čísla

Časově závislá chyba – konzument načte prázdnou hodnotu
– producent nezapiše do bufferu

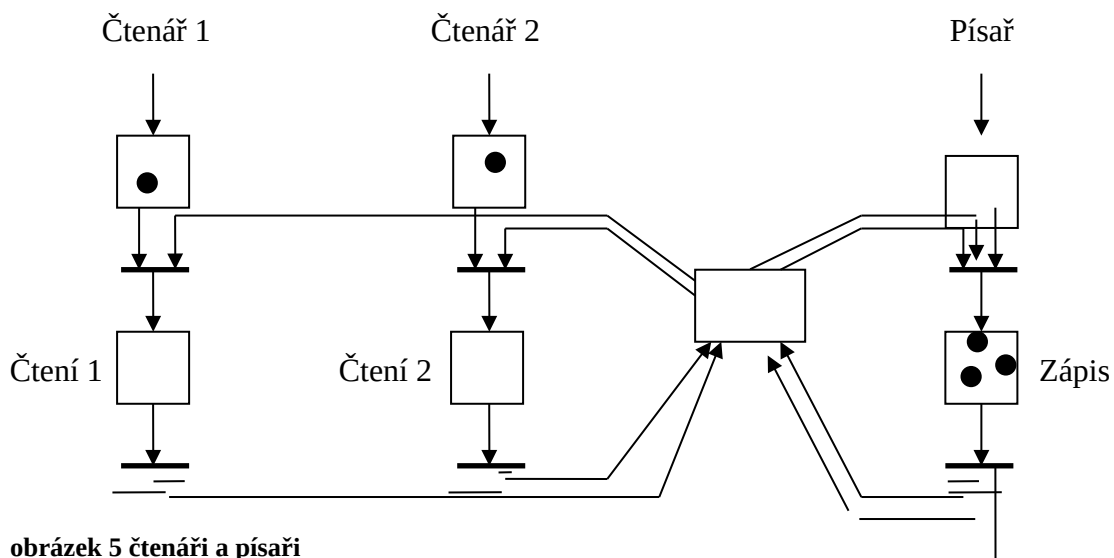
Čtenáři a písář

- Používá se v databázových systémech.
- **Písář** je proces, který může zapisovat do společné datové struktury.
 - Operace tohoto zápisu je jedinečná.
 - Je pouze jeden
- **Čtenář** je proces, který může číst.
 - Jeho operace je vícenásobná.
 - Je jich více
- V případě více čtenářů se musí zvětšit také počet teček.
-

Písář zapisuje – nelze číst

Alespoň jeden čtenář čte – nelze zapisovat

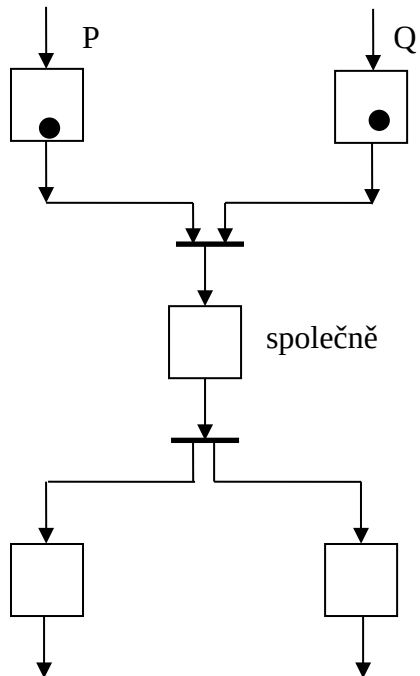
Číst může více čtenářů současně



obrázek 5 čtenáři a písáři

Souběh

- Používá se v distribuovaném prostředí.
- Jeden proces vyčká na druhý a společně provedou určitou část programu.
- Zabudován v jazyce ADA.



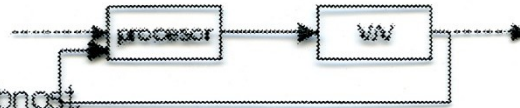
obrázek 6 souběh

1.2 Chyby v synchronizaci

Proč se používá multitasking?

- interaktivní zpracování – zkrácení doby odpovědi
- zvýšení stupně multitaskingu – zvýšení využití procesoru

- model



- pravděpodobnost,

že proces čeká na V/V,

tj. neběží

$$p = t_{vv} / (t_p + t_{vv})$$

že v tomto stavu se nachází

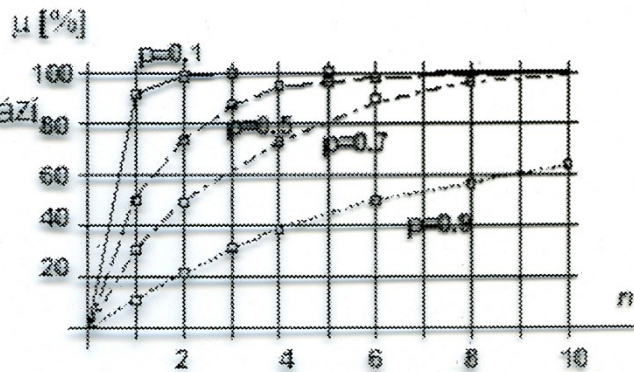
n procesů

$$p^n$$

a využití procesoru

n procesy

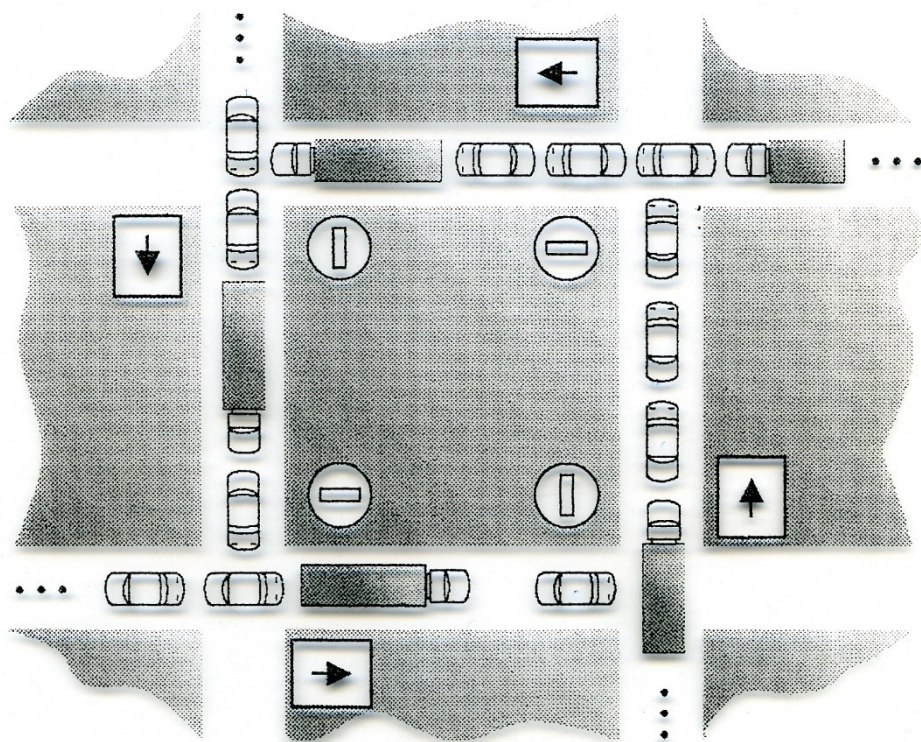
$$\mu = 1 - p^n$$



Stárnutí – proces příliš dlouho čeká (a nemusí se dočkat)

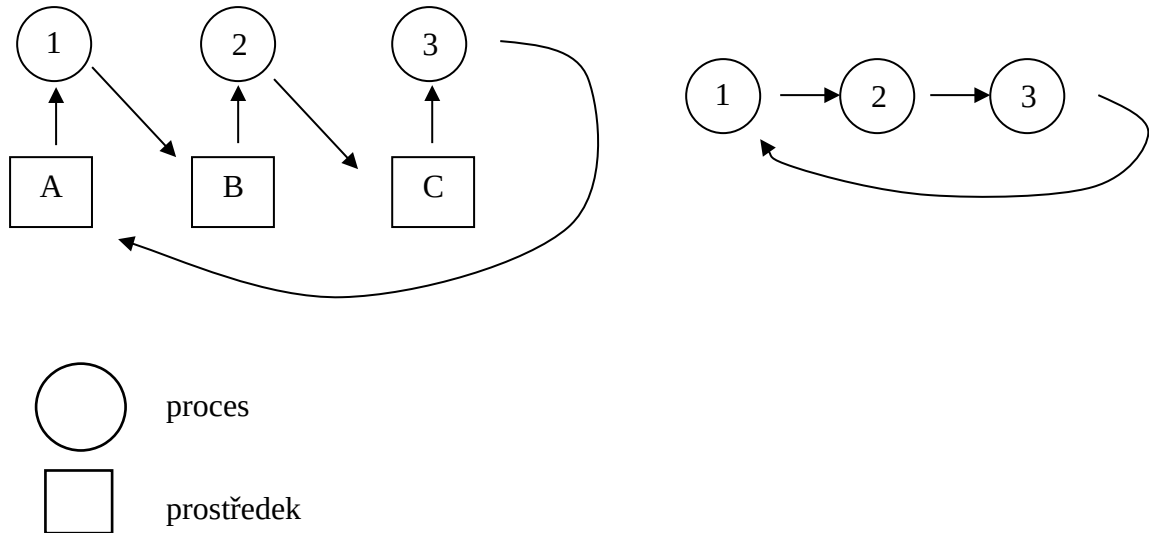
Uváznutí (deadlock)– proces je zablokovaný a nemůže již dále pokračovat.

Množina procesů uvázla, jestliže každý z procesů v této množině čeká na událost, kterou může vyvolat pouze některý z procesů v této množině.



Nutné podmínky uváznutí:

1. Vzájemná vylučnost – prostředek smí používat nejvýše 1 proces a je jich méně než procesů.
2. Inkrementální požadavky – proces vlastníci prostředky požaduje další.
3. Nepředbíhatelnost – prostředky přidělené procesu musí on sám vrátit, nikdo mu je nemůže odebrat.
4. Cyklické čekání – vytvoří se cyklický řetěz posloupností procesů z hlediska čekání na prostředky.



obrázek 7 nutné podmínky uváznutí

Ochrana před chybami

1. Můžeme v systému užít protokol, který zajistí, že *nikdy* deadlock nenastane. (prevence – zrušení jedné z podmínek 1-4, obcházení – zabránění současného uplatnění 1-4)
2. Můžeme systému dovolit, aby deadlock nastal a potom ho vyřešit (detekce a obnova).
3. Můžeme celý problém deadlocku ignorovat a předstírat, že k němu nikdy nedojde. Toto řešení zatím užívá většina OS, včetně Unixu („přstrosí“ politika).

1.3 Synchronizační nástroje

Semaforey

Používají se pro synchronizaci procesů pracujících nad stejnými daty. Základní operace jsou P (vstup) a V (výstup z kritické sekce).

Pokud jeden proces vstoupí do kritické sekce, nastaví se hodnota semaforu na 0 a ostatní procesy nemohou pokračovat. Semafor je proto řadí do fronty.

Při vystoupení procesu z kritické sekce se nastaví hodnota semaforu na nenulovou hodnotu. Do kritické sekce může vstoupit další proces. Přednost mají procesy ve frontě. Jejich vstupy a výstupy řídí semafor.

Řekněme, že máme mnoho spolupracujících procesů, které všechny čtou a zapisují záznamy

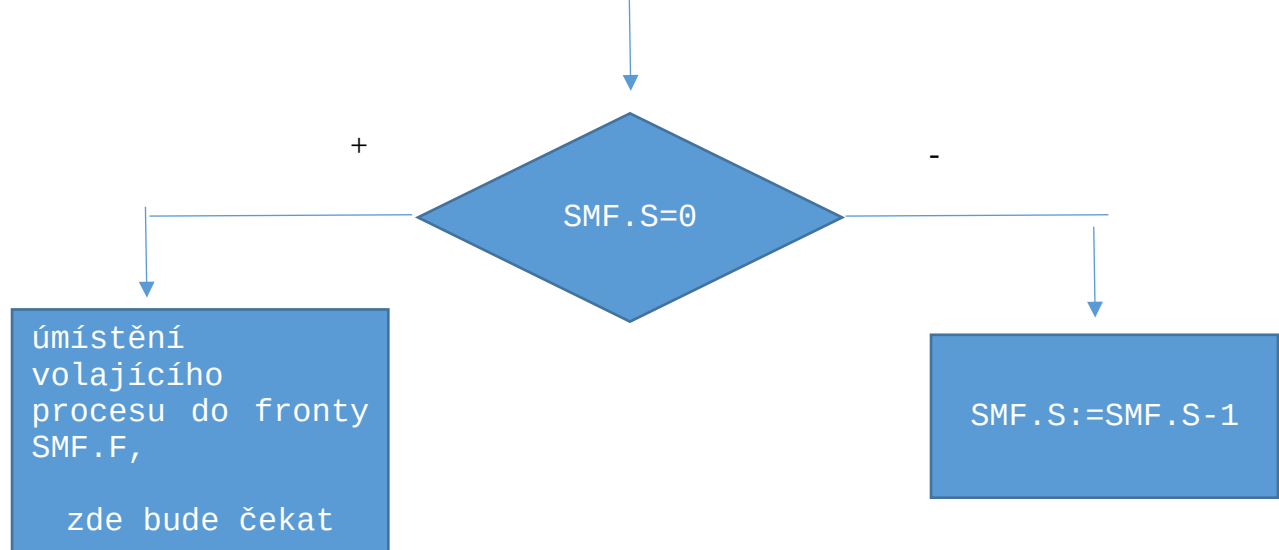
v jednom datovém souboru. Budeme zřejmě potřebovat přísně omezit přístup k souboru.

Můžeme použít semafor s počáteční hodnotou jedna a kolem kódu pro operaci se souborem umístit dvě semaforové operace: první bude testovat a dekrementovat hodnotu semaforu, druhá ji bude testovat a inkrementovat. První proces, který se pokusí o přístup k souboru, bude dekrementovat semafor a zdaří-li se mu to, bude mít semafor nově hodnotu 0. Tento proces nyní může pokračovat dále a používat datový soubor, pokud se však nyní pokusí o dekrementaci semaforu jiný proces, operace se nezdaří protože nová hodnota semaforu by byla -1. Druhý proces bude pozastaven do doby, než první proces inkrementuje hodnotu semaforu zpět na 1. Nyní je možno pozastavený proces probudit a tentokrát pokus o dekrementaci semaforu uspěje.

Každý objekt semaforu je popsán polem.

Příklad 1:

```
type Semafor = record S:word;  
                      F:fronta procesů;  
end;  
  
procedure Init_Sem(var SMF:Semafor;PH:word);  
begin  
    SMF.S:=PH;  
    SMF.F:={prázdná};  
end;  
  
procedure Wait(var SMF:Semafor);  
begin  
    if SMF.S=0 then {umístění volajícího procesu do fronty SMF.F,  
                   zde bude čekat}  
    else SMF.S:=SMF.S-1;  
end;
```

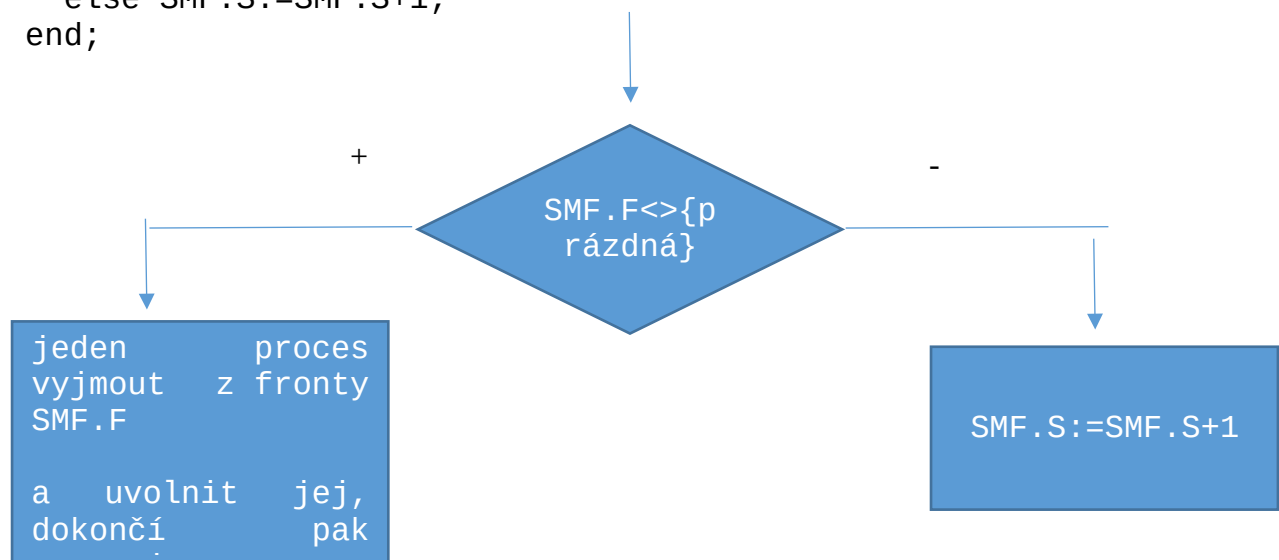


procedure Send(var SMF:Semafor);

begin

if SMF.F<>{prázdná} then {jeden proces vyjmout z fronty SMF.F a uvolnit jej, dokončí pak operaci Wait, ve které čeká}

else SMF.S:=SMF.S+1;
end;



Příklad 2:

Proces chce do kritické sekce

```
wait(Hldej);  
KSP;  
Send(Hldej);
```

Vzájemné vyloučení procesů

3 3
2 2
1 1
0 0

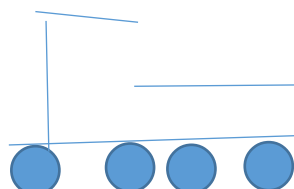
kritická sekce

producent

konzument

31

44



Producent – Konzument

```
const MaxPol=3;
var Buffer:array[0..MaxPol-1] of Typ_položky;
    Obsazene, Volne:Semafor;
    J, K:word;
    Vst_data, Vyst_data:Typ_položky;
begin
    J:=0;
    K:=0;
    Init_Sem(Obsazene,0);
    Init_Sem(Volne,MaxPol);
    cobegin
        -- Producent
        loop
            {Produkuje položku Vst_data}
            Wait(Volne);
            Buffer[J]:=Vst_data;
            J:=(J+1) mod MaxPol;
            Send(Obsazene);
        end loop;
        -- Konzument
        loop
            Wait(Obsazene);
            Vyst_data:=Buffer[K];
            K:=(K+1) mod MaxPol;
            Send(Volne);
            {Spotřebuje položku Vyst_data}
        end loop;
    coend;
end;
```

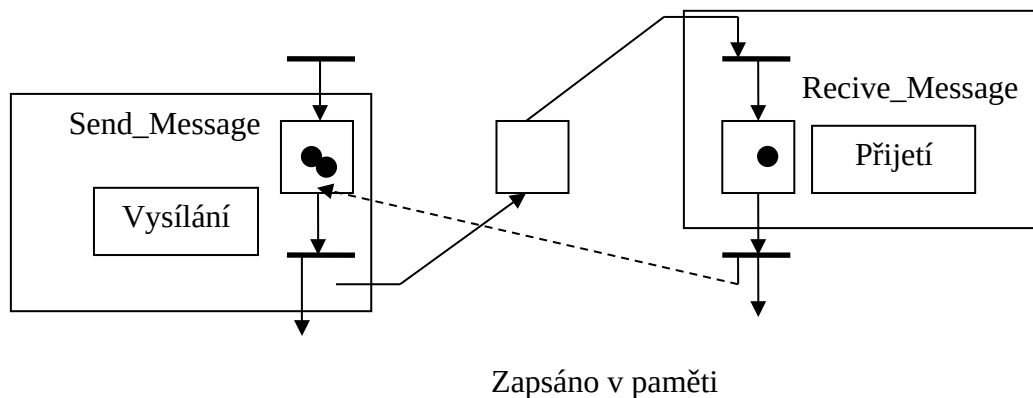

Zasílání zpráv

Při zasílání vnikají tyto operace: **send_Message(...)** – zaslání zprávy, kdy jeden proces zašle operaci (posloupnost bitů) – zpráva a druhý proces ji přijme **recive_Message(...)** – přijetí zprávy.

Adresace

- symetrická – uvádí se jméno přijímacího i vysílacího procesu
- asymetrická – uvádí se pouze jméno přijímacího procesu, jméno vysílajícího se určí až po přijetí zprávy
- nepřímá – uvádí se jméno vyrovnávací paměti nebo komunikačního kanálu

Asynchronní zasílání zpráv



obrázek 8 asynchronní zasílání zpráv

Při symetrické a asymetrické adresaci je vyrovnávací paměť poskytnuta systémovými prostředky (schránka). Může obsahovat i více zpráv, kapacita je ale omezena.

Příklad: Řešení úlohy Producent – Konzument pomocí asynchronního zaslání zpráv se symetrickou adresací.

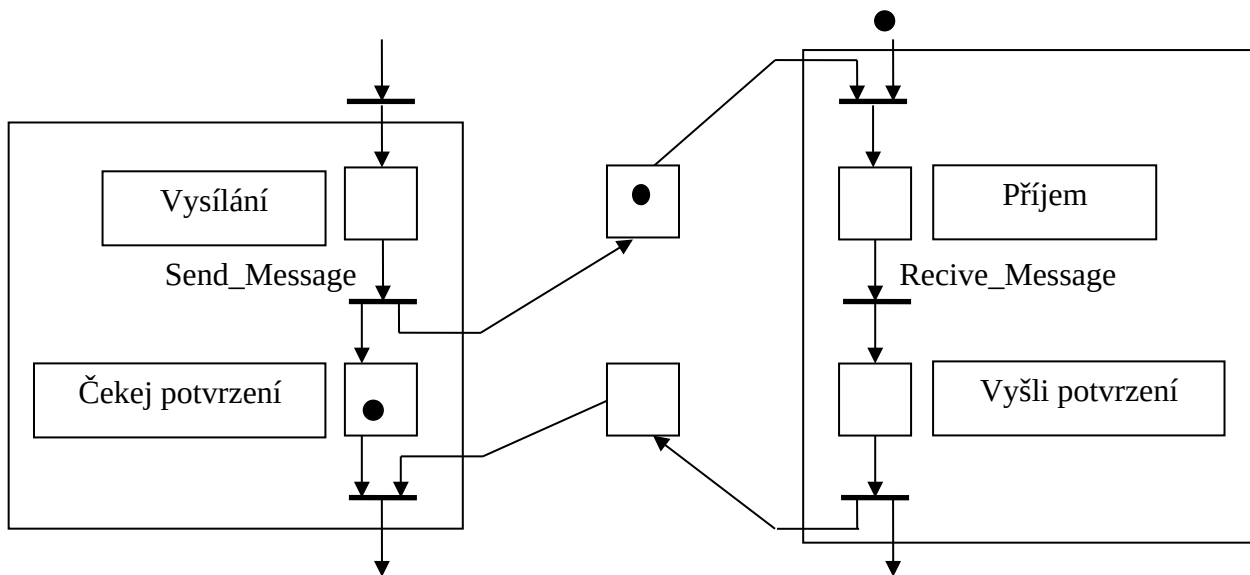
```
-- Producent – konzument:  
cobegin
```

```
  -- Producent;  
  declare P1:Typ_polozky;  
  begin  
    loop  
      Send_Message(Konzument, konverze na zprávu (P1));  
    end loop;  
  end Producent
```

```
-- Konzument
```

```
declare P2:Typ_polozky;  
begin  
  loop  
    Recive_Message(Producent, konverze na zprávu (P2));  
  end loop;  
end Konzument  
  
coend;
```

Synchronní zasílání zpráv



obrázek 9 synchronní zasílání zpráv

Monitory

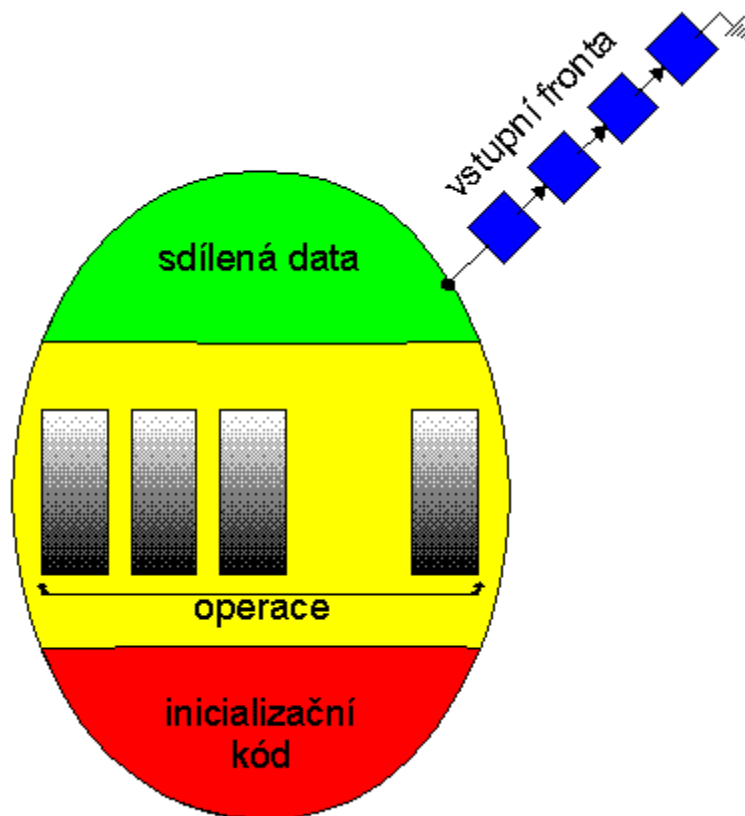
- Synchronizační nástroje obsažené ve vyšších programovacích jazycích, například MODULA.
- Monitor obsahuje deklaraci společné proměnné s definicí všech operací, které lze na ní provádět a jsou vzájemně vyloučeny.

```

M:monitor
  Cislo,jmeno;
  procedure PRG1(...);
  begin
    ...
  end;
  procedure PRG2(...);
  begin
    ...
  end;
  .
  .
  .
  procedure PPRG(...);
  begin
    ...
  end;

begin
  inicializace pr1...prm;
end;

```



Konstrukce monitoru zajišťuje, že pouze jeden proces může být v daném čase na monitoru aktivní, takže programátor se nemusí zabývat synchronizačním kódem. Je patrné, že definice monitoru je v mnoha ohledech podobná kritické oblasti a stejně jako existují podmíněné kritické oblasti, tak pro monitor je třeba definovat podmíněnost.